Metaheurística

Exercício 1 (Aula 2)

Análise Experimental de Busca Aleatória

Eric Takasumi

Março de 2026

1 Resultados Experimentais

1.1 Estatísticas Descritivas

Para a análise, foram realizados 30 experimentos independentes (utilizando sementes de 0 a 29) para cada função objetivo. A Tabela 1 resume o desempenho da Busca Aleatória Pura.

Tabela 1: Desempenho da Busca Aleatória Pura (30 execuções).

Função	**Média**	**Desvio Padrão**	**Melhor**	**Pior**
Esfera	7.5054	1.6829	4.8567	10.5732
Rastrigin	58.3415	5.8369	39.4577	66.9769

1.2 Curvas de Convergência

As Figuras 1 e 2 apresentam a evolução da convergência média com um intervalo de confiança (IC) de 95%.

O uso de um orçamento fixo de avaliações é fundamental para a validade estatística: como o cálculo do IC depende diretamente do tamanho da amostra (n), manter o mesmo número de observações em todos os pontos do eixo x garante que a margem de erro seja comparável ao longo de toda a execução. Caso o número de avaliações variasse, o IC tornaria-se instável ao final da curva, prejudicando a análise visual da estabilização do algoritmo.

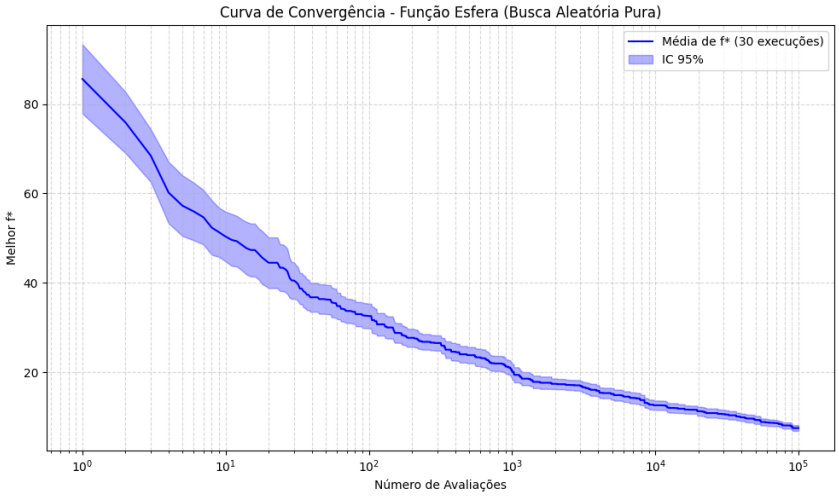

Figura 1: Curva de convergência para a função Esfera.

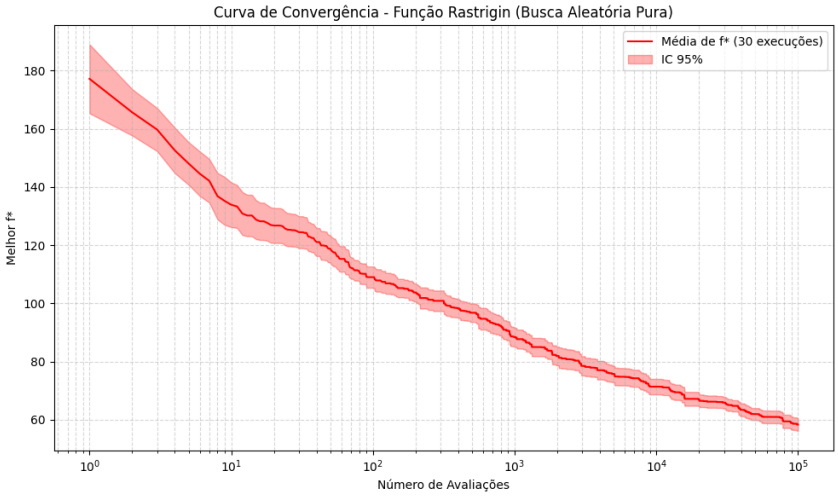

Figura 2: Curva de convergência para a função Rastrigin.

1.3 Análise e Relação com o Teorema No Free Lunch (NFL)

Os resultados evidenciam a disparidade na dificuldade de otimização entre as topologias. A função Esfera, por ser unimodal e estritamente convexa, apresenta uma ampla vizinhança de soluções aceitáveis, o que permite à Busca Aleatória Pura atingir valores moderados. Já a função Rastrigin, com sua topologia multimodal e altamente não linear, possui inúmeros ótimos locais que reduzem drasticamente a probabilidade de uma amostragem convergir para o ótimo global.

Segundo o Teorema *No Free Lunch* (NFL), não existe um algoritmo de propósito geral que supere todos os outros em todos os problemas possíveis. Este experimento ilustra esse princípio: a Busca Aleatória Pura, por carecer de mecanismos de explotação, apresenta um desempenho aceitável na Esfera mas falha na Rastrigin. Isso reforça que a eficiência de uma metaheurística está intrinsecamente ligada ao seu alinhamento com a estrutura do problema.

2 Código Python

```
# -*- coding: utf-8 -*-
"""Ex1_Aula2.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/13
       YZsGioyRvrRDg1U8HAIb0exxO3X9Pvh

Bibliotecas
"""

import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats

"""Defini  o das Fun  es de Otimiza  o"""

def fun_esfera(x): #Fun  o Esfera (unimodal)
    return np.sum(x**2, axis=1)

def fun_rastrigin(x): #Fun  o Rastrigin (multimodal)
    return np.sum((x**2) - 10 * np.cos(2 * np.pi * x) + 10, axis=1)

"""Busca aleat ria Pura"""

def busca_aleatoria(func, dim, limite, num_aval, seed):
    """
    Executa a busca aleat ria pura.

    Parametros:
    - func: fun  o a ser minimizada
    - dim: numero de vari veis (10)
    - limite: tupla com limite inferior e superior (-5.12, 5.12)
    - num_aval: numero de avaliacoes (100.000)
    - seed: semente para reprodutibilidade
    """
    # Registrando a semente
    np.random.seed(seed)

    low, high = limite

    # Gera todas as solu  es candidatas de uma vez
    X = np.random.uniform(low, high, size=(num_aval, dim))

    # Avalia as 100.000 solucoes
    avaliacoes = func(X)

    # Retorna o acumulado das menores solucoes encontradas
    return np.minimum.accumulate(avaliacoes)

"""Execu  o"""

dim = 10
limite = (-5.12, 5.12)
num_aval = 100000
```

```python
num_seeds = 29

# Sementes registradas (0 a 29)
seeds = list(range(0, num_seeds + 1))

resultados_esfera = []
resultados_rastrigin = []

print("Executando o experimento para 30 execu  es...")

for seed in seeds:

    historico_esfera = busca_aleatoria(fun_esfera, dim, limite,
        num_aval, seed)
    historico_rastrigin = busca_aleatoria(fun_rastrigin, dim, limite,
        num_aval, seed)

    resultados_esfera.append(historico_esfera)
    resultados_rastrigin.append(historico_rastrigin)

resultados_esfera = np.array(resultados_esfera)
resultados_rastrigin = np.array(resultados_rastrigin)

# Fun  o auxiliar para calcular e imprimir estat sticas
def print_stats(name, resultados):
    resultado = resultados[:, -1]

    media_val = np.mean(resultado)
    std_val = np.std(resultado, ddof=1) # ddof=1 para desvio padr o
        amostral
    melhor_val = np.min(resultado)
    pior_val = np.max(resultado) # O "pior" dos melhores resultados de
        cada run

    print(f"--- Resultados para a fun  o {name} ---")
    print(f"M dia (f*): {media_val:.4f}")
    print(f"Desvio Padr o (  ): {std_val:.4f}")
    print(f"Melhor resultado: {melhor_val:.4f}")
    print(f"Pior resultado: {pior_val:.4f}\n")

print_stats("Esfera", resultados_esfera)
print_stats("Rastrigin", resultados_rastrigin)

"""Plotagem dos Gr ficos com IC 95%"""

def plot_graph(name, results_matrix, color):
        mean_curve = np.mean(results_matrix, axis=0)
        std_curve = np.std(results_matrix, axis=0, ddof=1)

        n = results_matrix.shape[0]
        t_val = stats.t.ppf(0.975, df=n-1) # Valor cr tico da
            distribui  o t de Student
        ci = t_val * (std_curve / np.sqrt(n))

        plt.figure(figsize=(10, 6))
        x = np.arange(1, num_aval + 1)
```

```
        plt.plot(x, mean_curve, label=f'M dia de f* (30 execu  es)',
            color=color)
        plt.fill_between(x, mean_curve - ci, mean_curve + ci, color=
           color, alpha=0.3, label='IC 95%')

        plt.xscale('log')
        plt.xlabel('N mero de Avalia  es')
        plt.ylabel('Melhor f*')
        plt.title(f'Curva de Converg ncia - Fun  o {name} (Busca
           Aleat ria Pura)')
        plt.legend()
        plt.grid(True, which="both", ls="--", alpha=0.5)
        plt.tight_layout()
        plt.show()

print("Gerando os gr ficos de converg ncia...")
plot_graph("Esfera", resultados_esfera, 'blue')
plot_graph("Rastrigin", resultados_rastrigin, 'red')
```